

Q8) Find out the applications of recursion.
Is recursion still used in the present programming era. If so where?
What are the programming languages that support recursive calls.

* Applications of recursions are

1. Factorial calculation
2. Fibonacci series generation
3. Tree traversal
4. Graph traversal (DFS)
5. Binary search
6. Tower of Hanoi
7. Backtracking (Sudoku, N-Queens, Maze)
8. Divide and conquer (Merge sort, Quick sort)
9. File and directory traversal
10. Mathematical computations (GCD, power, sum of digits)

* Yes recursion is still widely used in modern programming, although developers use it where it makes the code simpler and more natural. In some cases, loops are preferred for better performance, but recursion remains essential in many areas.

Eg: file systems, web development, Artificial intelligence, DSA, graphs, compilers, sorting, mobile and desktop apps, games etc

Programming languages that support recursion

C, C++, Java, Python, JS, C#, Go, Rust, Swift, Kotlin

1. Find out the history of towers of Hanoi
2. Today from the morning 7:30 am to 4:30 pm list out various data structures in your surrounding

Data structure

Data structure is a method of organizing the large amount of data efficiently so that any operation on that data becomes easy.

Notes:

1. Every data structure is used to organize the large amount of data.
2. It follows a particular principle

Types of Data structures

1. Linear
2. Non-linear

* Linear: If a data-structure organizes the data in sequential order, then that data structure,

ex: arrays, list (linked list), stack, queue.

* Non-linear: It is a data structure in which elements are not stored sequentially and have one-to-many or many-to-many relationships.

Ans: The tower of Hanoi is a famous mathematical puzzle invented by Edouard Lucas in 1883.

Lucas introduced the puzzle under the name 'The Tower of Brahma' and described a legend about monks in an Indian temple.

According to the legend:

- There are three rods and 64 golden disks stacked on one rod, with the largest at the bottom and the smallest at the top.
- The monks move one disk at a time while following these rules:
 - a. Only one disk may be removed at a time.
 - b. A larger disk can never be placed on top of a smaller disk.
 - c. Only the top disk of any rod can be removed.
- The legend says that when all 64 disks are moved to another rod, the world will come to an end.
- Today, the tower of Hanoi is widely used in CS

and maths to teach recursion, algorithm design, and problem solving.

Minimum number of moves required: $2^n - 1$

$$2^3 - 1 = 7$$

2) • student attendance list - Array/List

- college bus queue - stack
- classroom seating - 2D Array
- Phone contacts - Hash Map
- college timetable - Array
- lunch in canteen - queue
- file folders - Tree
- social media - graph
- Browser back button - stack
- college campus roads - graph

• write a C++ code which includes a class student find out the space occupied by the object using `sizeof()`.

Algorithmic efficiency:

- Although computer scientists have studied algorithms and algorithm efficiency extensively, the field has not been given an official name.

• $f(n)$ = efficiency

Ans: when you use the sizeof operator on a class object, it returns the total memory occupied by that object in bytes.

```
#include <iostream>
using namespace std;
class student {
```

```
    int id; // 4 bytes
```

```
    char grade; // 1 byte
```

```
};
```

```
int main() {
```

```
    student s;
```

```
    cout << sizeof(s);
```

output: 8

⇒ By adding padding we get 8 (4 + 3 + 1)

manually = 5 (4 + 1)

Let us assume that :

1. It is a single processor machine
2. It is a 32 bit operating system machine
3. It performs sequential execution
4. It requires 1 unit of time for arithmetic and logical operations
5. It requires 1 unit of time for assignment and return value.
6. It requires 1 unit of time for read and write operations.

```
* int sum (int A[], int n) {
    int sum = 0, i;
    for (i = 0; i < n; i++) {
        sum = sum + A[i];
    }
    return sum;
}
```

int sumOfInt (int A[], int n) cost Repetation Total

int sum = 0, i;

for (i = 0; i < n; i++)

sum = sum + A[i]

return sum

1 + 1 + 1 1 + (n+1) + n 2n + 2

2 n 2n

1 1 1

4n + 4

i) code	cost	Repetition	Total
$i = 1$	1	1	1
$i < n$	1	n	n
$i++$	1	$n-1$	$n-1$
<code>printf("Hello")</code>	1	1	$\frac{1}{2n+1}$

ii) code	cost	repetition	Total
$i = 1;$	1	1	1
$i = i + 20;$	2	1	2
<code>printf("%d", i);</code>	1	1	$\frac{1}{4}$

iii) code	cost	repetition	Total
$i = 1;$	1	1	1
<code>while (i <= n) {</code>	1	$n+1$	$n+1$
<code>printf("%d", i);</code>	1	n	n
<code>i++ ; }</code>	1	n	n
			$\frac{3n+2}{4}$

2) Basic shell program: #!/bin/bash

echo -n "Enter 1st num?"

read num1

echo -n "Enter 2nd num?"

read num2

sum = \$(num1 + num2)

echo "The sum of num1 & num2 is \$sum"

code	cost	Repetition	Total
echo - n "enter 1st num"	1	1	1
read num1	1	1	1
echo - n "enter 2nd num"	1	1	1
read num2	1	1	1
sum = \$ ((num1 + num2)) (1 assign + 1 add)	2	1	2
echo "The sum of \$ num1 & \$ num2 is \$ sum"	1	1	1
			7